```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay

# 1. Load, Split & Standardize Dataset
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data[:, :2], iris.target, test_size=0.25, random_state=42, stratify=iris.target
)
scaler = StandardScaler()
X_train, X_test = scaler.fit_transform(X_train), scaler.transform(X_test)

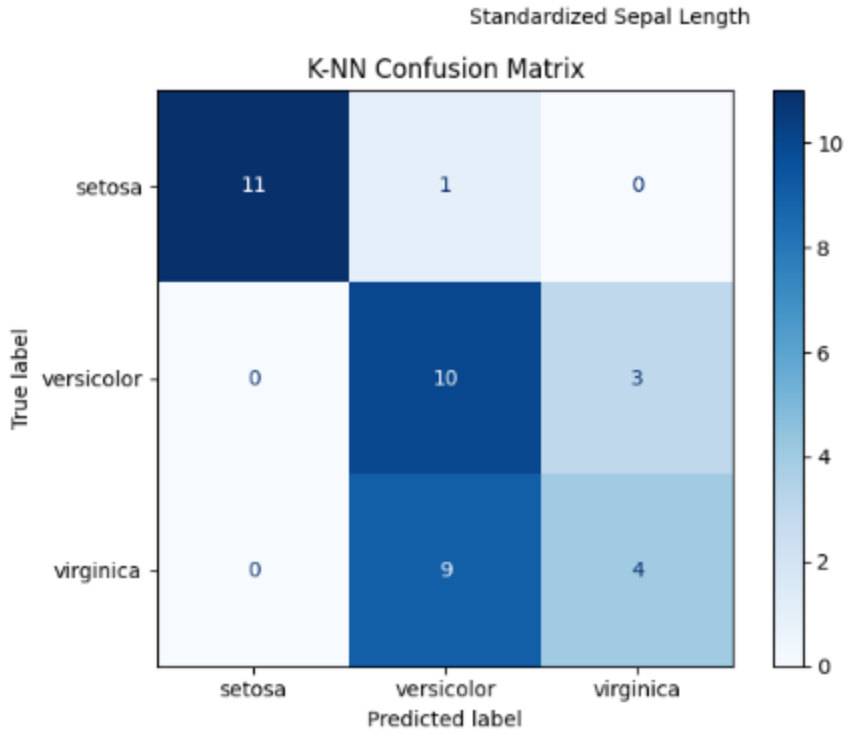
# 2. Train K-NN Model
model = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
y_pred = model.predict(X_test)

# 3. Print Evaluation Metrics
print("K-Nearest Neighbours Classification\n-----------------------------------\nNumber of neighbours (K): 5")
print(f"Test Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%\n\nSample predictions:")
for actual, predicted in zip(y_test[:5], y_pred[:5]):
    print(f"Actual: {iris.target_names[actual]}, Predicted: {iris.target_names[predicted]}")

# 4. Plot Decision Boundaries & Regions
x_min, x_max = X_train[:, 0].min() - 0.5, X_train[:, 0].max() + 0.5
y_min, y_max = X_train[:, 1].min() - 0.5, X_train[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.03), np.arange(y_min, y_max, 0.03))
```

```
Z = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)


plt.figure(figsize=(9, 5))
plt.contourf(xx, yy, Z, alpha=0.25, cmap="viridis")
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap="viridis", edgecolors="black", label="Training samples")
plt.xlabel
```




```
K-Nearest Neighbours Classification
-----------------------------------
Number of neighbours (K): 5
Test Accuracy: 65.79%

Sample predictions:
Actual: setosa, Predicted: setosa
Actual: versicolor, Predicted: virginica
Actual: versicolor, Predicted: versicolor
Actual: versicolor, Predicted: versicolor
Actual: setosa, Predicted: setosa
```

